

Beyond Dashboards

Building Observability into Modern Cloud Architectures

A simple guide to seeing what your system is doing
- and understanding why.

Emmanuela Oporum

Solutions Architect - Cloud, AI and Distributed Systems

THE QUESTION

Why did the user experience a problem?



Metrics

Something changed



Traces

The delay happened here



Logs

This event explains it

Everything looks green. But the customer cannot pay.

THE DASHBOARD SAYS

The service is healthy

Servers are running
Errors are low
CPU looks normal

THE CUSTOMER SAYS

My payment failed

The page kept loading.
I tried again.
No order was confirmed.

A healthy server does not always mean a healthy customer experience.

What we will make simple

- | | | |
|----|-----------------|---|
| 01 | Meaning | What observability means in plain English |
| 02 | Need | Why cloud systems are difficult to understand |
| 03 | Evidence | How logs, metrics and traces give us clues |
| 04 | Design | Where observability fits in an architecture |
| 05 | Action | How to start with one important user journey |

Observability explains why

Observability means collecting enough useful clues from a system so we can understand what it is doing - even when the problem is new.

THE MAIN QUESTION

What is happening, where is it happening, and why?

It is not one product. It is a way of designing and operating systems.

A dashboard shows selected answers

A dashboard answers questions that someone chose before the problem happened.

Useful dashboard questions

Is the service available?
How many requests failed?
Is CPU usage too high?
Is the queue getting full?

The unexpected question

Why are payments slow only for mobile users in one region after the latest release?

Dashboards help us see patterns. Observability helps us investigate new questions.

Monitoring warns. Observability explains.

MONITORING

A problem is happening

Payment failures are above the limit.
Response time is too slow.
The queue is almost full.

OBSERVABILITY

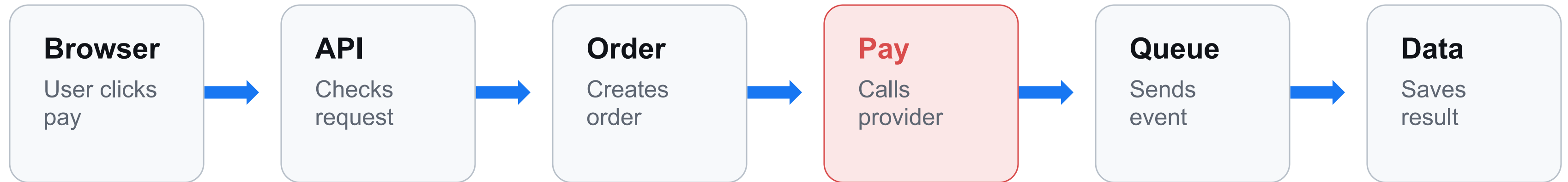
This is why it happened

The latest version created extra retries.
The payment provider became slow.
Only one customer path is affected.

We need both: fast warning and clear explanation.

One click crosses many services

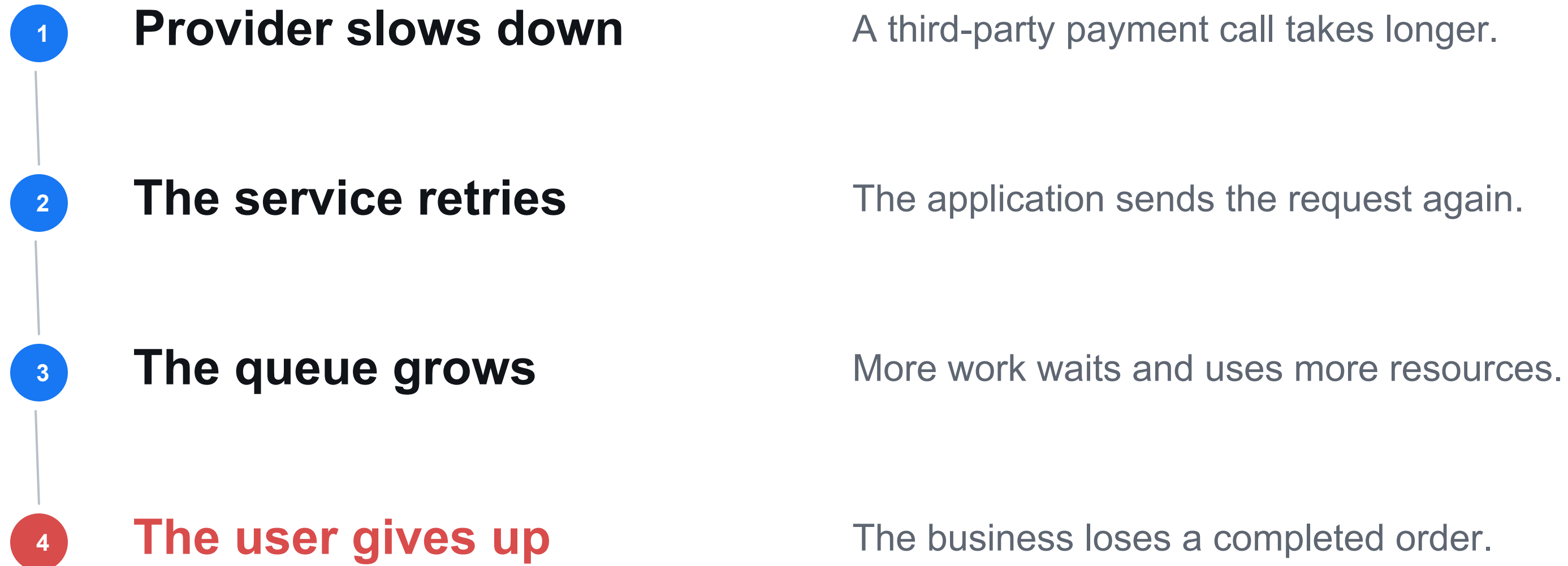
A modern cloud application is usually made of many connected parts.



The user sees one page. The team must understand the whole path.

A delay or failure in any part can change the final customer result.

A small problem can travel far



The first technical symptom and the final business impact may be far apart.

Think of a medical check

A doctor does not use one clue. A good investigation combines different clues.

METRICS

Measurements

Temperature, heart rate and blood pressure show that something changed.

LOGS

Detailed notes

Symptoms, events and test results explain what happened at a specific point.

TRACES

The journey

The order of steps shows where the problem began and how it moved.

One clue may warn us. Several connected clues help us understand.

Telemetry is the evidence a system produces

Telemetry is data created by an application or service that helps us understand its behaviour.

01

Logs

Records of events

02

Metrics

Numbers measured over time

03

Traces

The path of one request

Telemetry is the raw evidence. Observability is how well we can use it.

Logs record events

A log is a written record of an event inside the system.

```
12:04:18 payment_request_started
```

```
12:04:21 payment_provider_timeout
```

```
12:04:21 retry_attempt=2 order_id=8142
```

Logs can answer: What event happened? What failed? What details were recorded?

Useful logs are structured, clear and safe. They should not expose passwords or personal data.

Metrics show numbers over time

Metrics help us see size, speed, change and patterns.

1,250

Requests per minute

4.8%

Payment error rate

2.4 s

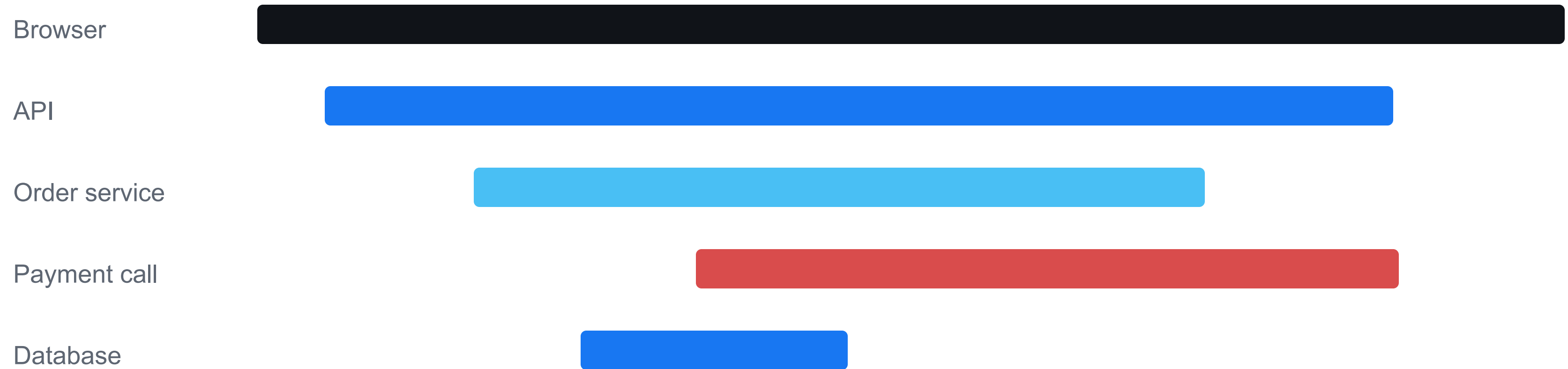
Slow-request time

Illustrative values

Metrics quickly show that something changed, but they may not explain the cause.

Trace one request

A trace shows every important step one request took across the system.



The payment step took most of the time.

A shared ID connects the clues

The same trace ID can connect a metric, a trace and the related logs.

SHARED TRACE ID

7f3a9b-c912

01

Metric

Error rate increased

02

Trace

Payment step was slow

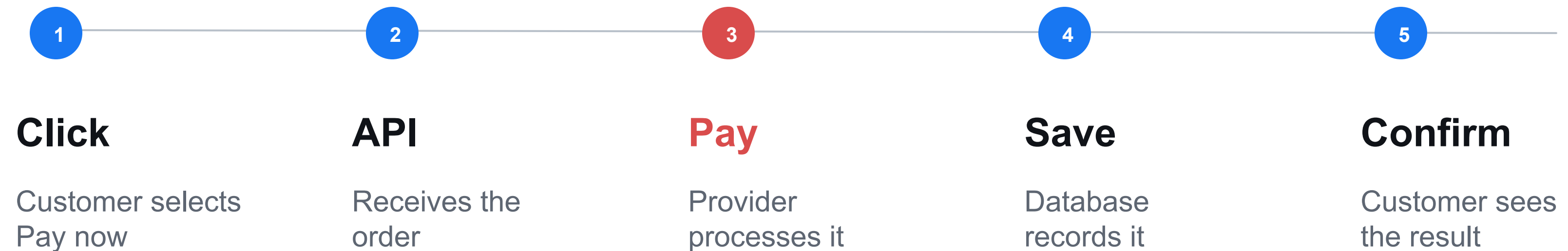
03

Log

Provider timed out

Without shared context, engineers must search and guess. With it, they can follow the evidence.

Follow the order journey



Observability should show whether the customer reached the final result - not only whether the API returned a response.

Start with the promise to the user

Do not begin with a tool. Begin with the result the user needs.

1 User journey

The customer can pay and receive a confirmed order.

2 Useful measure

How often is that journey successful and how long does it take?

3 Target

What level of reliability does the business promise?

Service measure: SLI

SLI = Service Level Indicator

Plain English: a number that tells us how the service is performing for the user.

Example

**Successful checkout rate =
completed orders / valid checkout attempts**

Choose a measure that reflects the user journey, not only the server.

An SLO is the reliability target

SLO = Service Level Objective

Plain English: the target we want the service to meet.

Illustrative target

99.9% of valid checkout attempts complete successfully each month

The right target depends on customer need, business risk and cost.

Four signals give a strong starting point

01

Latency

How long does it take?

02

Traffic

How much work arrives?

03

Errors

How much work fails?

04

Saturation

How full are limited resources?

These signals help us notice user impact and pressure before a larger failure.

Alerts should lead to action

An alert should mean: users are affected, or a serious impact is close.

NOISY ALERT

CPU changed

The chart moved, but nobody knows whether a person needs to act.

USEFUL ALERT

Checkout target is failing

Customers are affected and an engineer needs to investigate now.

Use alerts for action. Use dashboards for awareness and exploration.

Connected evidence shortens the search

WITHOUT CONNECTION

Search and guess

- Open several dashboards
- Search logs by time
- Ask which release changed
- Check each service separately
- Add more logs and wait

WITH CONNECTION

Follow one request

- Start from the user problem
- Open an affected trace
- Find the slow or failed step
- Open the connected logs
- Check the version and dependency

The goal is less guessing - not more screens.

A simple telemetry path

The application creates evidence. A collection layer prepares it. Tools store and analyse it.

1. Applications and services

Create logs, metrics and traces

2. Instrumentation

Adds useful names, context and trace IDs

3. Collector

Receives, filters, batches and sends telemetry

4. Storage and analysis

Dashboards, search, alerts and trace views

5. Team action

Investigate, fix, learn and improve

Record evidence at important boundaries

A boundary is a place where work moves from one part of the system to another.

01 User entry

Route, device type and journey

02 Service call

Dependency, attempt, timeout and result

03 Message queue

Message ID, delay, sender and receiver

04 Database

Operation, outcome and safe context

05 Third party

Provider, response time, retry and status

06 Business result

Order confirmed, payment accepted or file created

Safe telemetry

Telemetry can accidentally contain sensitive information. Decide what is allowed before production.

DO NOT RECORD

Passwords, full payment details, secret keys or private message content

LIMIT OR MASK

Email addresses, customer IDs and other personal information

SAFE TO USE

Service name, version, region, result, timing and controlled business fields

Collect what helps the investigation. Remove what creates unnecessary risk.

Keep useful evidence and reduce noise

Observability costs grow when systems create large amounts of low-value data.

KEEP

Errors, very slow requests, rare journeys and new releases

High investigation value

REDUCE

Healthy, repeated, high-volume events

Sample or keep for less time

REMOVE

Secrets, duplicates and fields nobody uses

No useful value or too much risk

Review the rules again when traffic, architecture or business risk changes.

Telemetry must be reliable

If telemetry stops during an incident, the team loses visibility when it needs it most.

1

Watch the collector

Check queue size, failed exports and dropped data.

2

Protect the application

Telemetry should not block normal customer work.

3

Plan for failure

Choose what waits, what is dropped and what creates an alert.

An observability problem should not become a customer outage.

Build it into delivery

- | | | |
|----|--------------|---|
| 01 | PLAN | Choose the user journey and questions |
| 02 | BUILD | Add useful evidence and shared context |
| 03 | TEST | Create a safe failure and check the signals |
| 04 | RUN | Alert on user impact and investigate |
| 05 | LEARN | Improve code, tests, notes and architecture |

Every incident should leave the system easier to understand next time.

Start with one important user journey

You do not need to change every system at once.

- | | | |
|----|----------------|--|
| 01 | Choose | Pick the user journey whose failure matters most. |
| 02 | Define | Write the user promise, measure and target. |
| 03 | Connect | Carry one trace ID across every service boundary. |
| 04 | Test | Create a safe failure and follow metric to trace to log. |
| 05 | Improve | Add the missing evidence and repeat with the next journey. |

Small scope. Complete evidence. Learn. Repeat.

**See the problem.
Understand the cause.
Improve the system.**

That is the value of observability.

Emmanuela Oporum

[LinkedIn: cloud-architect-emma](#)

Conf42 Observability 2026

Thank you